

Unidad 8

Calidad – Análisis de Resultados y Acciones Correctivas

1. Presentación

Esta unidad aborda el proceso que comienza **una vez ejecutadas las pruebas**, poniendo el foco no en *cómo testear* sino en **qué hacer con los resultados obtenidos**. Se trabaja la interpretación de evidencias, la clasificación y priorización de defectos, la propuesta de acciones correctivas y su verificación, entendiendo que el valor del testing no está en encontrar fallas, sino en **convertir hallazgos en mejoras concretas**.

El estudiante desarrollará criterios para analizar impacto, comunicar hallazgos con precisión, vincular defectos con sus causas probables y acompañar el ciclo de **corrección** → **re-testing** → **verificación** → **regresión** cuando corresponda. Además, se introducen prácticas de **trazabilidad**, **documentación útil**, **comunicación efectiva con desarrollo y mejora continua**, promoviendo una mirada donde la calidad es un proceso técnico, iterativo y colaborativo.

Se propone así comprender el rol de QA como una actividad que **fortalece el producto**, **mejora los procesos y consolida el trabajo en equipo**, integrando decisiones basadas en evidencia para sostener la calidad del software como un estándar de desarrollo.

2. Propósitos y objetivos

- **Interpretar y analizar resultados de pruebas**, comprendiendo su impacto en el producto y el proceso.
- **Clasificar y priorizar defectos** aplicando criterios de severidad, recurrencia y criticidad.
- **Proponer y documentar acciones correctivas** claras, medibles y orientadas a resultados.
- **Verificar correcciones mediante re-testing** y comprender el concepto de regresión como resguardo de estabilidad.
- **Aplicar trazabilidad y comunicación efectiva** como parte esencial del aseguramiento de calidad.
- **Promover la mejora continua** mediante decisiones basadas en evidencia, documentación y análisis técnico.
- **Ejercer el rol del QA con criterio crítico y responsabilidad**, entendiendo la calidad como proceso y no como etapa.

3. Contenidos

- **Del Testeo a la Calidad:** del hallazgo del defecto a la mejora del sistema; rol del QA, criterio e impacto.
- **Interpretación de resultados:** análisis de reportes más allá de PASS/FAIL; trazabilidad; diferencia entre error, defecto y falla.
- **Clasificación y priorización de defectos:** tipos de defectos, severidad vs prioridad, criterios de impacto y urgencia.
- **Acciones correctivas y re-testing:** propuesta de mejora, verificación de correcciones y concepto de regresión.
- **Comunicación QA → Desarrollo:** registro claro de incidencias, evidencia, pasos de reproducción y redacción orientada a soluciones.
- **QA y mejora continua:** PDCA aplicado al software, documentación útil y calidad como responsabilidad compartida.

4. Desarrollo teórico y práctico

4.1. Del testing a la calidad

Esta unidad parte de la premisa de que **ejecutar pruebas no finaliza el proceso**: la etapa posterior —interpretar resultados, priorizar defectos y ejecutar acciones correctivas— es donde se construye la calidad real del producto. Aquí no se profundiza en *cómo* diseñar cada tipo de prueba (eso corresponde a la Unidad 7); en cambio, el foco es **qué hacer con la evidencia** que generan las pruebas para mejorar el sistema y el proceso de desarrollo.

El flujo central que trabajaremos es:

Ejecución → Registro → Análisis → Corrección → Re-prueba.

Ejemplo: En una app de turnos médicos, una prueba detecta que es posible reservar dos turnos para el mismo médico y horario. El testing detecta el fallo, pero la calidad se construye cuando el equipo **analiza el impacto, prioriza la corrección y verifica que se resuelva sin generar nuevos problemas en el sistema.**

Reflexión: Pensá en un sistema cotidiano (banco, correo, tienda online). Cuando aparece un error, ¿qué consecuencias podría generar si **no se analiza su impacto, no se prioriza bien o no se vuelve a verificar la corrección?**

4.2. Interpretación y lectura de resultados

Interpretar un reporte de pruebas no se reduce a leer un estado *PASS* o *FAIL*; implica comprender el contexto en el que se ejecutó la prueba, la reproducibilidad del comportamiento observado, su impacto potencial y su relación con el requerimiento que debería satisfacerse.

La lectura de resultados es un proceso de análisis que transforma datos brutos en información útil para la toma de decisiones.

Un aspecto central de este análisis es la **trazabilidad**, entendida como la capacidad de conectar cada resultado con:

- a) el caso de prueba que lo generó,
- b) la evidencia que lo respalda (captura, log o reporte automatizado)
- c) la especificación o criterio de aceptación que permite determinar su validez.

En este marco se diferencian tres conceptos fundamentales:

- **Error**: acción humana que introduce una equivocación en el producto o proceso.
- **Defecto (bug)**: imperfección o falla interna en el software originada por un error.
- **Falla**: manifestación observable del defecto cuando el sistema se ejecuta.

Ejemplo: en una aplicación bancaria, un desarrollador escribe incorrectamente una condición lógica en el módulo de transferencias (**error**). Esa condición defectuosa se convierte en una validación fallida dentro del sistema (**defecto**), y finalmente el usuario experimenta que la app permite transferir montos negativos sin advertencia (**falla**). La secuencia muestra cómo un error introducido en el código puede propagarse hasta impactar en la experiencia real del usuario.

Comprender esta distinción es clave para documentar hallazgos con precisión, rastrear su origen y evitar confusiones entre causa, problema interno y efecto visible. La correcta interpretación de resultados no solo aporta rigor al proceso de QA, sino que permite priorizar acciones y fundamentar decisiones de corrección con criterio técnico.

4.3. Clasificación y priorización de defectos

La clasificación de defectos organiza los fallos según su naturaleza y facilita la identificación del área responsable de la corrección. Habitualmente se distinguen, entre otros, los siguientes tipos: **errores lógicos** (fallas en la estructura del razonamiento o condiciones del código), **errores de validación** (datos no controlados o ingresados en formato incorrecto), **errores de interfaz (UI/UX)** (problemas de presentación o interacción) y **errores de integración** (fallos en la comunicación entre módulos o servicios).

Paralelamente, la severidad describe el impacto técnico o funcional del defecto (Crítico / Alto / Medio / Bajo), mientras que la prioridad determina el orden en que se debe abordar la corrección (Alta / Media / Baja), resultado de combinar severidad con criterios de negocio — como la frecuencia de ocurrencia, el número de usuarios afectados y el costo de postergación.

Una **matriz Severidad × Frecuencia** es una herramienta operativa simple para transformar estas dimensiones en una prioridad accionable.

La priorización eficaz considera, además, elementos cualitativos: efectos sobre la integridad de datos, bloqueo de flujos críticos, consecuencias legales o de seguridad y riesgo de regresión al corregir.

Ejemplo breve: si en un sistema de facturación se detecta que el proceso admite facturas sin datos del cliente, la clasificación sería “error de validación” con severidad **Crítica** (riesgo de transacciones inválidas). Si además la frecuencia es alta, la matriz Severidad × Frecuencia deriva en prioridad **Alta**, lo que justifica una intervención inmediata y la ejecución de pruebas de regresión posteriores a la corrección.

Matriz Severidad x Frecuencia (Impacto / Ocurrencia)

Frecuencia \ Severidad	Crítico (sistema no funciona / pérdida de datos)	Alto (funcionalidad esencial afectada)	Medio (molestia significativa)	Bajo (cosmético o poco impacto)
Muy frecuente	Urgente (rojo) Intervención inmediata	Alta prioridad Corregir pronto	Planificado menor Corregir en el sprint	Baja prioridad Se agenda si hay tiempo
Frecuente	Alta prioridad No puede convivir con prod.	Media/Alta Arreglo en el corto plazo	Media Seguimiento en backlog	Baja Cosmético post release
Ocasional	Media/Alta Mitigar o diseñar fix	Media Revisar en iteración	Baja/Media Monitorear	Baja No bloquea roadmap
Rara vez	Media Arreglar si es replicable	Baja/Media Investigar caso borde	Muy baja Documentar hallazgo	Descartable Salvo acumulación de casos

Lectura rápida

- **Zona roja (Urgente):** algo que *se rompe mucho y rompe el sistema*.
- **Zona naranja (Alta):** afecta fuerte al usuario, sucede seguido, merece arreglo rápido.
- **Zona amarilla (Media):** molesto o riesgoso, pero planificable.
- **Zona verde/gris (Baja):** no altera el flujo central, se prioriza por acumulación o tiempo disponible.

4.4. Registro de resultados y evidencias

Registrar los resultados no solo documenta lo ocurrido durante una prueba, sino que permite reconstruir el historial completo de cada caso, garantizando trazabilidad, aprendizaje y responsabilidad técnica.

4.4.1. Propósito del registro

El registro cumple tres funciones esenciales:

- (1) controlar la ejecución de las pruebas registrando resultados esperados y obtenidos;
- (2) facilitar el análisis posterior para identificar patrones o áreas problemáticas;
- (3) proveer evidencias verificables que respalden las conclusiones del proceso de QA.

En síntesis: la evidencia convierte un hallazgo en dato accionable.

Aclaración operativa — Testing vs QA

La planilla de registro forma parte del **proceso de Testing** (ejecución y documentación de pruebas) y, al mismo tiempo, constituye un insumo esencial para **QA**, que la utiliza para analizar defectos, priorizarlos y dar seguimiento durante el ciclo de corrección. En síntesis:

Testing → registra evidencia. QA → usa la evidencia para decidir y mejorar.

4.4.2. Clasificación de resultados

Cada caso de prueba debe compararse entre resultado esperado y resultado obtenido, y clasificarse como: **Pass**, **Fail** o **Bloqueado**. Registrar también los *Pass* es necesario para mostrar cobertura y confirmar qué partes del sistema funcionan correctamente.

4.4.3. Tipos y características de la evidencia

Según corresponda, adjuntar capturas o breves videos, logs o trazas y/o reportes automáticos. Una evidencia efectiva debe ser **clara**, **reproducible** y **verificable** (fecha e identificada con el caso de prueba).

4.4.4. Formato mínimo obligatorio de registro

Para homogeneizar prácticas y facilitar la evaluación, cada registro deberá completar la siguiente tabla mínima (campos exigidos en planilla o ticket):

- **ID** (p. ej. TP8-01)
- **Caso de prueba** (título / objetivo)
- **Resultado esperado**

- **Resultado obtenido**
- **Estado** (Pass / Fail / Bloqueado)
- **Evidencia** (archivo o enlace)
- **Severidad propuesta** (Crítico / Alto / Medio / Bajo)
- **Prioridad propuesta** (Alta / Media / Baja)
- **Propuesta de corrección** (breve)
- **Responsable asignado**
- **Fecha de registro**

(Formato recomendado; incluir estas columnas facilita la evaluación y la trazabilidad.)

Ejemplo:

ID: TP8-03

Caso de prueba: Reservar turno — mismo médico y horario

Resultado esperado: Sistema impide reserva duplicada con mensaje “Horario no disponible”

Resultado obtenido: El sistema permite crear la segunda reserva y muestra confirmación

Estado: Fail

Evidencia: captura_turno_dup_20251109.png (link)

Severidad propuesta: Alto

Prioridad propuesta: Alta

Propuesta de corrección: Agregar validación en backend que verifique solapamiento de reservas; devolver 409 si existe conflicto.

Responsable asignado: Dev-Backend (María)

Fecha de registro: 2025-11-09

(Explicación breve: la severidad “Alto” se propone porque impide la experiencia correcta del usuario, y la prioridad “Alta” porque el flujo principal de la app (reserva) queda comprometido. La propuesta indica la ubicación técnica y el criterio de aceptación.)

4.4.5. Soportes y herramientas

Los registros pueden mantenerse en planillas o en trackers (Jira, GitLab Issues, Trello o hasta un documento compartido el Google Docs).

La herramienta es secundaria: lo esencial es respetar el **formato mínimo** y adjuntar evidencia verificable.

4.5. Tipos de defectos y severidad (clasificación y priorización)

4.5.1. Tipos de defectos

- **Errores lógicos:** fallas en la lógica o en condiciones (ej.: cálculo erróneo).
- **Errores de validación:** datos no validados (ej.: formulario que acepta email inválido).
- **Errores de interfaz (UI/UX):** problemas de presentación o interacción.
- **Errores de integración:** fallas en la comunicación entre módulos o servicios.

Estos tipos ayudan a derivar la incidencia al equipo responsable (desarrollo, diseño, datos, integración).

4.5.2. Severidad vs prioridad

Aunque suelen confundirse, **severidad** y **prioridad** no son lo mismo y responden a preguntas distintas:

Concepto	Responde a...	Lo define...	Es estable o variable
Severidad	<i>¿Qué tan grave es el fallo para el sistema o la funcionalidad?</i>	El impacto técnico o funcional del defecto	Tiende a ser estable (no cambia salvo que cambie el alcance del problema)
Prioridad	<i>¿Cuándo conviene resolverlo en relación a otros defectos o tareas?</i>	Urgencia organizacional y de negocio	Es dinámica (puede cambiar según contexto, fechas, riesgo, stakeholders)

Severidad (Impacto técnico/funcional)

Clasifica **el daño que causa el defecto** dentro del sistema:

- **Crítico:** Bloquea el sistema o destruye datos. No existe solución alternativa viable.
- **Alto:** Rompe una función esencial, pero el sistema sigue operativo parcialmente.
- **Medio:** Afecta la experiencia o provoca errores no bloqueantes.
- **Bajo:** Cosmético o periférico; no compromete el uso correcto del sistema.

La severidad habla del software, no del negocio.

Prioridad (Decisión de orden de resolución)

Determina **cuándo debe corregirse**, combinando múltiples factores:

- **Severidad** (¿qué tan grave es?)
- **Frecuencia** (¿ocurre todo el tiempo o raramente?)
- **Impacto de negocio** (¿afecta ingresos, reputación, clientes clave?)
- **Costo de postergación** (¿qué pasa si no se arregla ahora?)
- **Restricciones contextuales** (fechas de entrega, demo a cliente, normativas, etc.)

“La prioridad habla del negocio, del momento y de la estrategia, no del bug en sí.”

Relación clave (lo que más suele confundirse)

Situación	¿Puede ocurrir?	Ejemplo
Alta severidad, baja prioridad	Sí	Un fallo crítico que solo sucede en un módulo interno que no usa nadie por 6 meses.
Baja severidad, alta prioridad	Sí	Un error cosmético en el botón de pago de la web, visible para miles de usuarios.
Alta severidad y alta prioridad	Sí	El sistema no guarda ventas y ocurre todos los días.
Baja severidad y baja prioridad	Sí	Un texto levemente desalineado en una pantalla poco visitada.

Mini-ejemplo

En un sistema de gestión académica, se detecta que al exportar certificados en PDF, **los acentos no se muestran correctamente.**

- Desde el punto de vista **técnico/funcional**, no rompe el proceso: el PDF se genera igual → **Severidad: Baja.**
- Pero el sistema está a días de imprimir **500 certificados oficiales** que van a entregarse en un acto institucional → **Prioridad: Alta**, porque el daño reputacional y el costo de reprocesar son enormes.

Conclusión: la severidad no obligó a arreglarlo, pero el contexto de negocio sí lo volvió urgente.

Un defecto no se corrige antes por ser técnicamente grave, sino porque **el contexto lo vuelve impostergable.**

4.5.3. Matriz y criterios de priorización

Para decidir la prioridad se aplicará una matriz sencilla (Severidad × Frecuencia) y reglas complementarias. Procedimiento operativo:

1. **Asignar severidad** según impacto técnico/funcional:
 - Crítico: pérdida de datos, bloqueo de procesos esenciales, riesgo legal/seguridad.
 - Alto: funcionalidad principal afectada, gran degradación de UX.

- Medio: desajustes que afectan flujos secundarios o generan trabajo extra.
 - Bajo: problemas estéticos o de usabilidad sin impacto funcional.
2. **Estimar frecuencia:** número estimado de usuarios o transacciones afectadas (Alta / Media / Baja).
 3. **Definir prioridad** con la matriz:
 - Severidad **Crítica** → Prioridad **Alta** (independiente de frecuencia).
 - Severidad **Alto** + Frecuencia **Alta/Media** → Prioridad **Alta**.
 - Severidad **Alto** + Frecuencia **Baja** → Prioridad **Media**.
 - Severidad **Medio** + Frecuencia **Alta** → Prioridad **Media**.
 - Severidad **Medio/Bajo** + Frecuencia **Baja** → Prioridad **Baja**.
 4. **Reglas de excepción:** si la corrección implica riesgo de introducir regresiones en módulos críticos, considerar una tarea de corrección urgente solo para cambios de alta prioridad; cambios cosméticos se agrupan para releases mayores.

Ejemplo resuelto:

Error que permite emitir factura sin cliente → Severidad **Crítica** (riesgo de transacciones inválidas) → Prioridad **Alta** → corrección antes del release.

4.6. Análisis de causa raíz, corrección y re-pruebas

4.6.1. Análisis de causa raíz (ACR)

El **Análisis de Causa Raíz (ACR)** es un proceso estructurado que busca responder **por qué ocurrió un defecto**, no solo describir *qué falló*. Su objetivo no es culpar a personas, sino encontrar **la causa más profunda y corregirla de forma que el error no vuelva a repetirse**.

A diferencia de la corrección inmediata (*apagar el incendio*), el ACR apunta a **evitar futuros incendios**.

Dos enfoques simples y muy usados

1. Los 5 porqués (5 Whys)

Consiste en hacer preguntas consecutivas del tipo *¿por qué?* hasta llegar a la causa raíz lógica.

Ejemplo:

- El sistema duplicó un pago. **¿Por qué?**
- Porque el usuario pudo presionar dos veces el botón sin restricción. **¿Por qué?**
- Porque no hay un bloqueo temporal del botón tras el primer clic. **¿Por qué?**
- Porque no estaba contemplado en los criterios de aceptación. **¿Por qué?**

- Porque el requerimiento priorizó rapidez de desarrollo y no incluyó controles anti-doble-envío.

Causa raíz: *Requerimiento incompleto (sin controles de idempotencia).*

Acción correctiva adecuada: incorporar la prevención desde el requerimiento y agregar bloqueo, debounce o token anti reenvío.

2. Diagrama de Ishikawa (Espina de Pescado)

Permite organizar visualmente posibles causas agrupándolas en categorías frecuentes en software:

- **Requerimientos**
- **Diseño**
- **Implementación (código)**
- **Datos**
- **Infraestructura / Integraciones**
- **Proceso / Testing**



Ejemplo aplicado al mismo problema del pago duplicado:

Categoría	Posible causa detectada
Requerimientos	No se especificó control anti doble envío
Diseño	No se propuso un mecanismo de idempotencia
Implementación	No se deshabilita el botón tras el primer clic
Testing	No se incluyó un caso de prueba para envíos repetidos
Integración	La API procesa ambos eventos sin validación

En este ejemplo, **varios factores contribuyen**, pero la **causa raíz se ubica en el requerimiento**, porque si hubiese estado definido, todas las etapas posteriores lo habrían contemplado.

Qué diferencias tiene corregir síntomas vs. causa raíz

Enfoque	Resultado típico	¿Evita recurrencia?
<i>Corregir el síntoma</i> (ej: borrar pagos duplicados manualmente)	Solución temporal	No
<i>Corregir la causa raíz</i> (agregar validación y ajustar requerimientos y tests)	Sistema más robusto	Sí

Idea clave de esta sección: Un defecto que vuelve a aparecer nunca estuvo resuelto; solo estuvo mitigado. El ACR transforma correcciones reactivas en mejoras permanentes del proceso.

4.6.2. Propuesta y documentación de la corrección

La acción correctiva debe describir: **qué** se cambia, **por qué**, **criterio de aceptación** y **responsable**. Toda corrección se registra en el ticket con la evidencia asociada.

4.6.3. Re-testing y pruebas de regresión

Tras la corrección se ejecuta el re-testing del caso afectado. Si la corrección puede afectar áreas relacionadas, se ejecutan pruebas de regresión definidas previamente. Los resultados del re-testing se registran con el mismo nivel de detalle que la prueba original.

Pruebas de regresión

Consisten en volver a ejecutar pruebas previamente exitosas para asegurarse de que una corrección o modificación **no haya generado nuevos defectos** en otra parte del sistema. El objetivo no es verificar la solución puntual del error, sino **confirmar que el sistema completo sigue funcionando como antes**.

Idea clave: Corregir un defecto puede introducir otros. La regresión busca detectar esos efectos secundarios.

Ejemplo:

Se corrige un error en el cálculo de un precio en el carrito. Después de aplicar la corrección, se ejecutan nuevamente pruebas de login, agregar productos, aplicar descuentos, medios de pago y confirmación de compra. Si algo que antes funcionaba ahora falla → se produjo regresión.

Diferencia rápida entre Re-Testing y Regresión

Concepto	Pregunta que responde	Alcance
Re-testing	¿El defecto que corregimos fue resuelto?	Puntual (solo el caso que falló)
Regresión	¿La corrección rompió otra cosa?	Global (impacto en otras funciones)

Re-testing confirma la **corrección**. Regresión confirma la **no ruptura**.

4.6.4. Flujo y estados del ciclo de incidencia

Flujo recomendado y campos mínimos en cada ticket:

- **Abierto:** incidencia reportada con evidencia mínima. (Campos obligatorios: ID, Caso, Resultado esperado/obtenido, Evidencia, Severidad propuesta).
- **En revisión:** QA revisa reproducibilidad y ajusta severidad/prioridad; puede pedir información adicional.
- **Asignado:** responsable técnico acepta la tarea. Se añade estimación de esfuerzo.
- **Corregido:** desarrollador indica cambios realizados y sube evidencia (commit/link).
- **Re-probado:** QA ejecuta re-testing y registra resultado.
- **Cerrado o Reabierto:** si re-testing ok → cerrado; si falla → reabierto con nueva evidencia.

Ejemplo de redactado de ticket:

Título: [TP8-03]

Reserva duplicada — mismo médico y horario

Descripción: Al intentar reservar un turno duplicado el sistema confirma la segunda reserva.

Pasos para reproducir: 1) Login usuario; 2) Seleccionar médico X; 3) Seleccionar fecha 2025-11-09; 4) Reservar turno 10:00; 5) Repetir reserva 10:00 → Resultado observado: segunda reserva confirmada. Resultado esperado: sistema debe impedir segunda reserva y mostrar “Horario no disponible”.

Severidad propuesta: Alto.

Evidencia: captura_turno_dup_20251109.png.

Propuesta corrección: validar solapamiento en backend y responder 409. Responsable propuesto: Dev-Backend.

4.7. Aseguramiento de la calidad (QA) y mejora continua — enfoque teórico y ejemplos

El Aseguramiento de la Calidad (QA) complementa al testing con un enfoque preventivo y sistémico: se centra en fortalecer procesos, validar requisitos y promover prácticas que reduzcan la aparición de defectos.

QA no es únicamente un conjunto de técnicas, sino un marco de trabajo que incorpora **criterio técnico, documentación y responsabilidad compartida** para sostener la calidad en el tiempo.

4.7.1. PDCA aplicado a correcciones

El ciclo **PDCA (Plan → Do → Check → Act)** es una herramienta útil para transformar hallazgos de testing en mejoras sostenibles:

- **Plan (Planificar):** a partir del reporte de defectos, definir el objetivo de la corrección (qué se corregirá, impacto esperado, criterios de aceptación) y los indicadores que permitirán evaluar el éxito.
- **Do (Hacer):** implementar la corrección en un entorno controlado, documentando los cambios (commits, pull requests, notas de implementación).
- **Check (Verificar):** verificar mediante re-testing y, cuando corresponda, suites de regresión limitadas, si la corrección cumple los criterios de aceptación. Registrar los resultados de verificación.
- **Act (Actuar):** si la verificación es satisfactoria, integrar la corrección a la rama principal y actualizar documentación / criterios; si no, volver a análisis de causa raíz y replantear la acción.

4.7.2. Métricas e indicadores útiles (qué medir y por qué)

Algunas métricas que permiten convertir datos de incidencias en insumos para la mejora continua:

- **Defectos por módulo** (cantidad y tendencia): identifica áreas de riesgo.
- **Tiempo medio de resolución (MTTR)**: mide la capacidad de respuesta del equipo.
- **Tasa de re-open** (incidencias reabiertas): indica problemas en la calidad de las correcciones.
- **% de Pass en re-testing**: verifica la efectividad de las acciones correctivas.

Estas métricas deben interpretarse en contexto y utilizarse para priorizar intervenciones en procesos o en la arquitectura del sistema, no como fines en sí mismas.

4.7.3. Prácticas y roles (cómo aporta cada actor a la QA)

La QA efectiva implica responsabilidades distribuidas:

- **Analista / Product Owner**: clarificar criterios de aceptación y asegurar que los requerimientos sean verificables.
- **Desarrollo**: implementar correcciones con trazabilidad (commits vinculados a tickets) y pruebas unitarias mínimas.
- **Tester / QA**: documentar evidencia, evaluar reproducibilidad y validar re-testing.
- **Equipo**: aplicar revisiones entre pares (code review) y registrar lecciones aprendidas.

Estas prácticas fomentan la prevención (baja reincidencia de defectos) y fortalecen la colaboración entre roles.

4.7.4. Integración conceptual con la unidad

En el marco de esta unidad, QA se presenta como **el paso lógico que sigue a la detección de defectos**: no basta con identificar fallos, es necesario analizarlos, priorizarlos, corregirlos y verificar que la corrección produzca la mejora esperada sin causar regresiones.

La identificación de defectos adquiere su sentido pleno cuando se integra en un ciclo de mejora: detección (testing), análisis (QA), acción (corrección) y verificación (re-testing y regresión). QA desplaza la pregunta de “qué falló” a “cómo evitar que vuelva a fallar”, transformando evidencia puntual en criterios y procesos que elevan la madurez del desarrollo.